Snorkel

fabricenduwayo

⚠ **Please do not reach out to reviewers directly;** messaging them will not speed up your review.

Difficulty Guidelines

Task difficulty is determined by accuracy when run against frontier AI models. This guide helps you design tasks at the right difficulty level.

Difficulty Levels

Difficulty is calculated from accuracy across two evaluation models. The threshold that applies depends on the tier:

DIFFICULTY	THRESHOLD
Hard	Accuracy ≤ 20% on the **best** model, **OR** accuracy ≤ 20% on the **wor**
Medium	20% < accuracy ≤ 60% on the **worst** model
Easy	60% < accuracy ≤ 80% on the **worst** model

> **Important:** Tasks where the worst model scores **above 80%** will not be accepted — they're too easy to be useful as training signal.

Why "best" vs "worst" model?

Both models are run against every task. The **worst model** sets the difficulty floor for most tasks: if even the weaker model can solve it most of the time, the task is Easy. The **best model** matters for the hardest tasks: a task where the strongest model still only scores ≤ 20% earns Hard difficulty even if the

worst model also struggles, because the failure isn't just a weak-model artifact.

Evaluation Process

Each task is evaluated against:

- **GPT-5.5** with Codex agent
- **Claude Opus 4.8** with Claude Code agent
- **5 runs each** to determine average accuracy

Designing for Difficulty

For Hard Tasks (≤ 20% on best or worst model)

Hard tasks require one or more of:

- **Deep domain expertise** — Knowledge LLMs haven't seen much
- **Complex multi-step reasoning** — 10+ sequential steps
- **Subtle debugging** — Root cause analysis required
- **Niche tools/languages** — Less common technologies

Techniques:

- Use bespoke rules buried in common patterns
- Require understanding of obscure documentation
- Create debugging tasks where the root cause isn't obvious
- Use domain-specific knowledge (blockchains, scientific computing)

For Medium Tasks (20–60% on worst model)

Medium tasks typically involve:

- **Moderate complexity** — 5-10 steps
- **Some domain knowledge** — Common but not trivial
- **Clear requirements** — But non-obvious solution

Techniques:

- Combine multiple familiar concepts
- Add edge cases that require careful handling
- Include configuration that's easy to miss

For Easy Tasks (60–80% on worst model)

Easy tasks should still be:

- **Non-trivial** — Not one-liners
- **Multi-step** — At least 3-5 steps
- **Testable** — Clear success criteria

Techniques:

- Standard tasks with one or two tricky aspects
- Well-known problems with specific constraints
- Setup tasks with multiple requirements

Common Mistakes

Making Tasks Too Easy

MISTAKE	IMPACT
Single-step solutions	Agents solve immediately
Obvious debugging	Pattern matching succeeds
Common tutorials	In training data
Simple API usage	Well-documented

Making Tasks Unfair

MISTAKE	IMPACT
Impossible requirements	No one can solve
Ambiguous instructions	Luck determines success
Time-dependent	Results vary randomly
External dependencies	Environment issues

Testing Your Difficulty

Before submitting, verify difficulty by:

1. Run Against Oracle Agent

BASH Copy

```
harbor run -a oracle -p <task-folder>
```

This should PASS. If it doesn't, your task may have issues.

2. Run Against Real Agents

BASH Copy

```
# GPT-5.5
stb harbor run -m @openai/gpt-5.5 -p <task-folder>

# Claude Opus 4.8
stb harbor run -m @anthropic/claude-opus-4-8 -p <task-folder>
```

Run at least 2-3 times against each model to gauge pass rate, and remember that the **worst** model's pass rate is what determines Easy/Medium

for most tasks.

3. Analyze Failures

When agents fail:

- Is it for a **good reason** (task difficulty)?
- Or a **bad reason** (unclear instructions)?

Good failures: Agent misunderstood complexity, missed edge case Bad failures: Ambiguous requirements, environment issues

Adjusting Difficulty

To Make Harder:

- Add more steps
- Include hidden requirements
- Use niche knowledge
- Create debugging scenarios
- Add edge cases

To Make Easier:

- Reduce step count
- Make requirements more explicit
- Use common technologies
- Provide more hints in instructions
- Simplify the environment

Next Steps

- Start creating your task
- Review CI checks

PREVIOUS
NEW: Instruction Prompt Styling

NEXT
Example Tasks